

Sección 1: Preparar la arquitectura del agente y procesos SDLC (15-20%)

Esta sección evalúa la capacidad de integrar agentes de IA en el ciclo de vida de desarrollo de software (SDLC), definir los límites entre planificación y ejecución, y configurar la observabilidad y el control necesarios para operar agentes autónomos de forma segura dentro del ecosistema de GitHub.

1.1 Integrar agentes en el ciclo de vida de desarrollo de software (SDLC)

Conceptos clave

¿Qué es un agente en el contexto del SDLC?

Un agente de IA es un sistema autónomo o semiautónomo que puede **percibir** su entorno, **razonar** sobre la información disponible y **actuar** para alcanzar un objetivo definido. En el contexto del SDLC dentro del ecosistema de GitHub, un agente opera como un **contribuidor más del equipo**: recibe tareas, crea ramas, escribe código, abre Pull Requests y responde a revisiones, todo de forma programática.

El modelo mental clave es: **tratar la salida de un agente como si fuera el Pull Request de un desarrollador junior**. Esto significa que:

- Toda salida del agente debe pasar por revisión de código.
- El agente no tiene permisos para hacer merge directo a **main**.
- Las protecciones de rama, CODEOWNERS y checks de CI aplican igual que para cualquier humano.

Identificación de pasos del SDLC para que los agentes los realicen

No todas las fases del SDLC son candidatas ideales para la automatización con agentes. La clave está en identificar **tareas repetitivas, bien definidas y verificables**:

Fase del SDLC	Tareas candidatas para agentes	Ejemplo en GitHub
Planificación	Desglose de issues, creación de sub-tareas	Copilot analiza un issue y sugiere un plan de implementación
Diseño	Generación de esqueletos de código, scaffolding	Copilot coding agent crea la estructura inicial del proyecto
Implementación	Escritura de código, refactorización	Copilot coding agent crea una rama y escribe el código
Testing	Generación de tests unitarios, tests de integración	El agente genera tests y los ejecuta vía GitHub Actions

Fase del SDLC	Tareas candidatas para agentes	Ejemplo en GitHub
Revisión	Revisión automatizada de PRs, análisis de calidad	Copilot code review analiza el diff y deja comentarios
Despliegue	Scripts de deployment, configuración de pipelines	GitHub Actions orquestado por el agente
Mantenimiento	Corrección de bugs, actualizaciones de dependencias	Dependabot + Copilot coding agent para fixes

Criterio para decidir si un paso es apto para un agente:

1. **Determinismo parcial:** ¿El resultado esperado es predecible y verificable?
2. **Bajo riesgo:** ¿Un error del agente puede revertirse fácilmente?
3. **Feedback rápido:** ¿Hay mecanismos automáticos (CI/CD) que validen el resultado?
4. **Definición clara:** ¿Las entradas y salidas están bien especificadas?

Identificación y mitigación de antipatronos comunes en agentes

Los antipatronos son prácticas que parecen razonables pero que conducen a resultados pobres o riesgosos:

Antipatrón	Descripción	Mitigación
Sobre-autonomía	Dar al agente permisos para actuar sin supervisión (merge directo, despliegue sin aprobación)	Usar branch protection rules, required reviews y CODEOWNERS
Sin validación	No ejecutar CI/CD sobre el trabajo del agente	Configurar GitHub Actions con checks obligatorios en los PRs del agente
Omitir la fase de planificación	Dejar que el agente pase directamente a escribir código sin un plan estructurado	Configurar el agente para generar un plan antes de actuar y requerir aprobación humana del plan
Tratar al agente como infalible	Asumir que el código generado es correcto sin revisión	Aplicar el mismo proceso de code review que para cualquier contribuidor humano
Scope creep del agente	El agente intenta resolver más de lo que se le pidió, modificando archivos no relacionados	Definir claramente los límites del issue y usar instrucciones precisas
Falta de trazabilidad	No poder reconstruir por qué el agente tomó una decisión particular	Configurar logging, artefactos inspeccionables y mantener el historial en el PR

Antipatrón	Descripción	Mitigación
Retroalimentación ignorada	El agente no aprende de las revisiones previas	Usar archivos de instrucciones personalizadas (.github/copilot-instructions.md) para codificar aprendizajes

Definición de entradas, salidas y criterios de éxito para agentes

Para que un agente funcione correctamente, debe tener un **contrato claro**:

Entradas (Inputs):

- El issue o tarea asignada (título, descripción, criterios de aceptación)
- El contexto del repositorio (código existente, convenciones, dependencias)
- Instrucciones personalizadas ([.github/copilot-instructions.md](#))
- Archivos de referencia (documentación, esquemas, tests existentes)

Salidas (Outputs):

- Una rama con los cambios de código
- Un Pull Request con descripción clara del cambio
- Tests que validen la funcionalidad
- Artefactos de CI/CD (logs, reportes de cobertura)

Criterios de éxito:

- Los checks de CI pasan (build, lint, tests)
- El PR cumple con las reglas del repositorio (CODEOWNERS, revisiones requeridas)
- El cambio resuelve el issue original
- No introduce regresiones (verificado por tests)
- El código sigue las convenciones del proyecto

Aplicación práctica en GitHub

El flujo del Copilot Coding Agent

1. Se crea/asigna un Issue → El agente lo analiza
2. El agente crea una rama (ej: `copilot/fix-123`)
3. El agente escribe código y hace commits
4. El agente abre un Pull Request
5. GitHub Actions ejecuta CI (build, tests, lint)
6. CODEOWNERS son notificados para revisión
7. Revisores humanos aprueban o solicitan cambios
8. Si hay cambios solicitados → el agente los aplica
9. Tras aprobación + checks verdes → merge

Configuración del repositorio para trabajo con agentes

```
# .github/copilot-instructions.md
# Instrucciones personalizadas para el agente Copilot

## Convenciones del proyecto
- Usar TypeScript estricto
- Tests unitarios con Jest para toda función nueva
- Seguir el patrón de arquitectura hexagonal
- Documentar funciones públicas con JSDoc

## Restricciones
- No modificar archivos en /config/production/
- No agregar dependencias sin justificación en el PR
- Máximo 300 líneas por archivo
```

```
# Ejemplo de branch protection rules (configurado en Settings > Branches)
# - Require pull request reviews: 1 revisor mínimo
# - Require status checks: CI/build, CI/test, CI/lint
# - Require CODEOWNERS review
# - No permitir push directo a main
```

Puntos clave para el examen

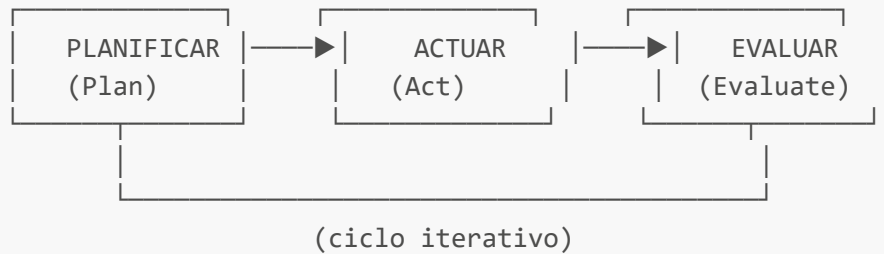
- ☒ Los agentes deben tratarse como **contribuidores junior**: su trabajo siempre pasa por revisión.
- ☒ GitHub es el **sistema de registro (system of record)** y el **plano de control** para la actividad del agente.
- ☒ Las **branch protection rules, required reviews** y **CODEOWNERS** son los mecanismos de gobernanza principales.
- ☒ El antipatrón más peligroso es la **sobre-autonomía**: dar al agente permisos para actuar sin validación humana.
- ☒ Siempre definir **entradas claras** (issue bien descrito), **salidas esperadas** (PR con tests) y **criterios de éxito** (CI verde + revisión aprobada).
- ☒ El archivo `.github/copilot-instructions.md` permite personalizar el comportamiento del agente por repositorio.
- ☒ GitHub Actions es el mecanismo principal para **validar automáticamente** el trabajo del agente.

1.2 Definir límites entre la planificación, el razonamiento y la acción

Conceptos clave

El ciclo de vida del agente: Plan → Act → Evaluate

Todo agente que opera en el SDLC sigue un ciclo fundamental de tres fases:



Planificar (Plan): El agente analiza el problema, descompone la tarea en pasos y genera un plan estructurado antes de hacer cualquier cambio.

Actuar (Act): El agente ejecuta el plan: escribe código, crea archivos, ejecuta comandos.

Evaluar (Evaluate): Se verifica si la acción cumplió los objetivos. Esto puede ser automático (CI) o humano (code review).

La separación de estas fases es **crítica** porque permite:

- Revisar el plan antes de que el agente actúe.
- Detectar errores de razonamiento tempranamente.
- Mantener control sobre qué hace el agente y cuándo.

Configurar la planificación del agente de forma independiente de su ejecución

La planificación y la ejecución deben ser **procesos desacoplados**. Esto significa que:

1. **El agente genera un plan** → se almacena como un artefacto revisable.
2. **Un humano revisa y aprueba el plan** → solo entonces el agente puede proceder.
3. **El agente ejecuta el plan aprobado** → sus acciones están acotadas por lo planeado.

¿Por qué separar planificación de ejecución?

- **Control de riesgos:** Un plan mal concebido detectado antes de la ejecución ahorra tiempo y evita errores en producción.
- **Trazabilidad:** Si algo sale mal, se puede rastrear si el problema fue en la planificación o en la ejecución.
- **Gobernanza:** Diferentes personas pueden aprobar planes vs. aprobar código final.

Implementación en GitHub:

```
Issue creado
|
▼
Copilot Coding Agent analiza el issue
|
▼
El agente publica un comentario en el issue con el plan propuesto
|
▼
```

```
Revisor humano valida el plan (aprueba o sugiere cambios)
|
▼
Tras aprobación → El agente crea rama y ejecuta el plan
|
▼
PR abierto → CI + Code Review → Merge
```

Configuración de un agente para generar un plan estructurado

Un plan estructurado debe contener:

1. **Análisis del problema:** Qué se necesita resolver y por qué.
2. **Archivos afectados:** Lista de archivos que se crearán, modificarán o eliminarán.
3. **Pasos de implementación:** Secuencia ordenada de cambios.
4. **Riesgos identificados:** Posibles problemas y su mitigación.
5. **Criterios de verificación:** Cómo se comprobará que el plan se ejecutó correctamente.

Ejemplo de plan estructurado generado por un agente:

```
## Plan de implementación: Issue #42 – Agregar autenticación OAuth

### Análisis
El sistema actual no tiene autenticación. Se necesita integrar OAuth 2.0
con GitHub como proveedor de identidad.

### Archivos afectados
- `src/auth/oauth.ts` (nuevo)
- `src/middleware/auth.ts` (nuevo)
- `src/routes/login.ts` (nuevo)
- `src/app.ts` (modificar – agregar middleware)
- `tests/auth/oauth.test.ts` (nuevo)

### Pasos
1. Crear módulo de configuración OAuth
2. Implementar middleware de autenticación
3. Crear rutas de login/callback/logout
4. Integrar middleware en la app principal
5. Escribir tests unitarios y de integración

### Riesgos
- Manejo incorrecto de tokens → Mitigación: usar librería probada (passport.js)
- Exposición de secrets → Mitigación: usar GitHub Secrets, no hardcodear

### Verificación
- Tests unitarios pasan
- Flujo OAuth completo funciona en entorno de desarrollo
- No se exponen credenciales en el código
```

Validar planes de agentes

La validación de un plan puede ocurrir en varios niveles:

Nivel	Qué se valida	Quién valida	Mecanismo en GitHub
Automático	Formato del plan, archivos mencionados existen, no toca archivos protegidos	GitHub Actions	Workflow que analiza el plan
Humano	Lógica del enfoque, riesgos, diseño	Revisor designado	Comentario de aprobación en el issue
Organizacional	Cumplimiento de políticas, seguridad	CODEOWNERS / Security team	Rulesets y políticas de repositorio

Preguntas para validar un plan:

- ¿El plan aborda todos los requisitos del issue?
- ¿Los archivos afectados son los correctos?
- ¿El plan introduce riesgos de seguridad?
- ¿El plan sigue las convenciones del proyecto?
- ¿Los criterios de verificación son suficientes?

Impedir la acción del agente hasta que el plan esté comprobado y aprobado

Este concepto es fundamental: el agente **no debe ejecutar ninguna acción** (escribir código, crear ramas, hacer commits) hasta que su plan haya sido revisado y aprobado por un humano autorizado.

Mecanismos para implementar esta restricción:

1. Gate de aprobación en GitHub Actions:

```
# .github/workflows/agent-gate.yml
name: Agent Plan Approval Gate
on:
  issue_comment:
    types: [created]

jobs:
  check-approval:
    if: contains(github.event.comment.body, '/approve-plan')
    runs-on: ubuntu-latest
    steps:
      - name: Verify approver is authorized
        run: |
          # Verificar que quien aprueba tiene permisos
          echo "Plan aprobado por ${github.event.comment.user.login}"
      - name: Trigger agent execution
```

```
run: |  
  # Señalizar al agente que puede proceder  
  echo "Agente autorizado para ejecutar el plan"
```

2. Environments con protección en GitHub Actions:

- Crear un environment llamado `agent-execution`
- Configurar "Required reviewers" en ese environment
- El workflow del agente debe referenciar ese environment
- El agente no puede proceder hasta que un revisor apruebe

3. Branch protection + revisiones requeridas:

- Aunque el agente cree una rama y un PR, no puede hacer merge sin aprobación.
- Esto actúa como una barrera final, pero idealmente la barrera debe estar antes (en el plan).

Aplicación práctica en GitHub

Ejemplo completo del flujo Plan → Aprobación → Ejecución

```
[Paso 1] Desarrollador crea Issue #42 con requisitos claros  
      ↓  
[Paso 2] Copilot Coding Agent es asignado al issue  
      ↓  
[Paso 3] El agente analiza el repo y genera un plan como comentario:  
         "Propongo los siguientes cambios: ..."  
      ↓  
[Paso 4] Desarrollador revisa el plan:  
         - Si OK → comenta "/approve-plan"  
         - Si NO → comenta con sugerencias, agente regenera plan  
      ↓  
[Paso 5] Tras aprobación, el agente:  
         - Crea rama `copilot/fix-42`  
         - Implementa los cambios según el plan  
         - Abre PR referenciando el issue  
      ↓  
[Paso 6] GitHub Actions ejecuta CI automáticamente  
      ↓  
[Paso 7] CODEOWNERS reciben notificación para code review  
      ↓  
[Paso 8] Tras aprobación del PR + CI verde → merge a main
```

Configuración de Rulesets para control de agentes

Los **Repository Rulesets** (la evolución de branch protection rules) permiten definir políticas granulares:


```
Ruleset: "agent-governance"
├─ Target: ramas que coincidan con "copilot/*"
├─ Reglas:
│   ├── Require pull request before merging
│   │   ├── Required approving reviews: 1
│   │   ├── Require review from CODEOWNERS: ✓
│   │   └── Dismiss stale reviews: ✓
│   ├── Require status checks
│   │   ├── CI/build: required
│   │   ├── CI/test: required
│   │   └── CI/lint: required
│   └── Block force pushes: ✓
└─ Bypass: ninguno (ni siquiera admins)
```

Puntos clave para el examen

- ☒ El ciclo **Plan** → **Act** → **Evaluate** es el modelo fundamental de operación de agentes en el SDLC.
- ☒ La **planificación debe estar desacoplada de la ejecución**: el agente no debe actuar sin un plan aprobado.
- ☒ Un plan estructurado incluye: análisis, archivos afectados, pasos, riesgos y criterios de verificación.
- ☒ Los **Environments con reviewers requeridos** en GitHub Actions son un mecanismo clave para implementar gates de aprobación.
- ☒ Los **Rulesets** de repositorio permiten definir políticas granulares para ramas creadas por agentes.
- ☒ La validación de planes ocurre en tres niveles: automático (CI), humano (revisores) y organizacional (políticas).
- ☒ Impedir la acción sin aprobación es un principio de seguridad, no una limitación: **protege al equipo de errores costosos**.

1.3 Configurar la observabilidad y el control de los agentes autónomos

Conceptos clave

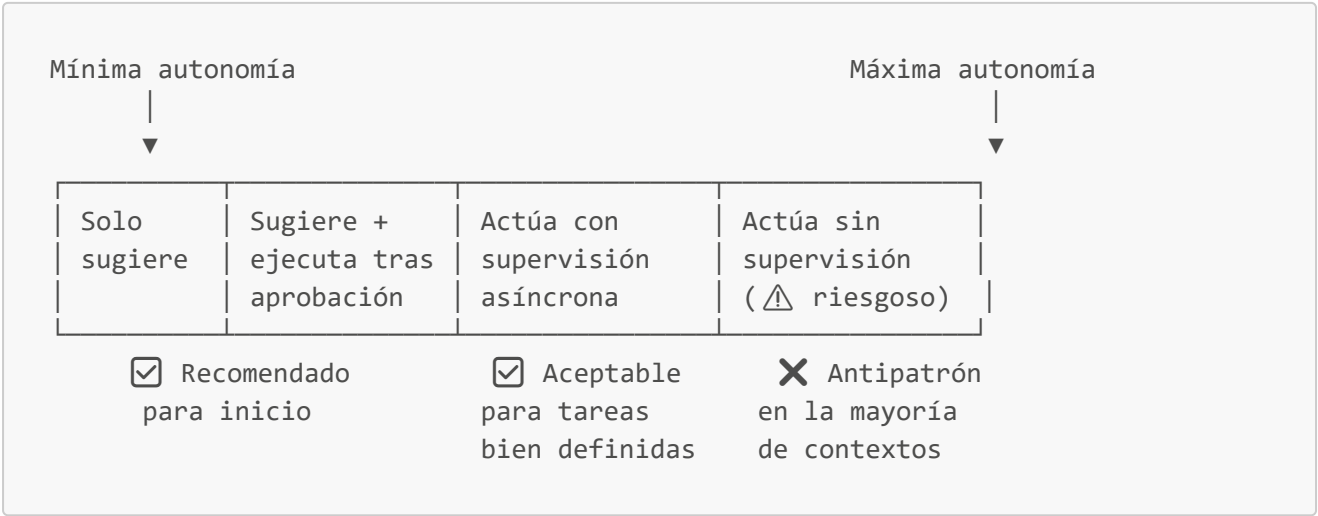
¿Qué es la observabilidad en el contexto de agentes?

La observabilidad es la capacidad de **entender qué está haciendo un agente, por qué lo está haciendo y cuál fue el resultado**, usando señales externas como logs, métricas y artefactos. En el ecosistema de GitHub, la observabilidad se logra a través de:

- **Pull Requests**: Registro completo de los cambios propuestos por el agente.
- **Commits**: Historial granular de cada modificación.
- **GitHub Actions logs**: Registro de la ejecución de CI/CD sobre el trabajo del agente.
- **Comentarios en Issues/PRs**: Registro del razonamiento y decisiones del agente.
- **Artefactos de Actions**: Archivos generados durante la ejecución (reportes, logs detallados).

Grado de autonomía del agente y límites de protección (guardrails)

El grado de autonomía define **cuánto puede hacer un agente sin intervención humana**. Se define en un espectro:



Límites de protección (guardrails) son las restricciones que acotan lo que el agente puede hacer:

Guardrail	Implementación en GitHub	Propósito
Branch protection	Settings > Branches > Protection rules	Impedir cambios directos en ramas protegidas
Required reviews	Mínimo N aprobaciones antes de merge	Garantizar supervisión humana
CODEOWNERS	Archivo CODEOWNERS en el repositorio	Asegurar que los dueños del código revisen cambios en sus áreas
Required status checks	CI debe pasar antes de poder hacer merge	Validación automática del trabajo del agente
Rulesets	Settings > Rules > Rulesets	Políticas granulares por patrón de rama
Environment protection	Environments con reviewers requeridos	Gates de aprobación para despliegues
Restricciones de permisos	Tokens con scope mínimo necesario	Principio de mínimo privilegio
Instrucciones personalizadas	.github/copilot-instructions.md	Guiar y restringir el comportamiento del agente

Planear e implementar el grado de autonomía

Para definir el grado de autonomía adecuado, considerar:

- 1. **Impacto del error:** ¿Qué pasa si el agente se equivoca?
 - Bajo impacto (typo en documentación) → más autonomía aceptable.

- Alto impacto (cambio en API pública, migración de BD) → más supervisión requerida.

2. **Reversibilidad:** ¿Se puede deshacer el cambio fácilmente?

- Un commit se puede revertir → más autonomía.
- Un despliegue a producción no siempre → menos autonomía.

3. **Madurez del equipo:** ¿Qué tan cómodos están con agentes?

- Equipos nuevos en agentes → empezar con autonomía mínima.
- Equipos experimentados → pueden aumentar autonomía gradualmente.

4. **Cobertura de tests:** ¿Hay tests que detecten regresiones?

- Alta cobertura → más confianza en dar autonomía.
- Baja cobertura → más riesgo, más supervisión.

Ejemplo de matriz de autonomía por tipo de tarea:

Tarea	Autonomía	Guardrails requeridos
Corrección de typos	Alta	CI + 1 reviewer
Generación de tests	Alta	CI + 1 reviewer
Refactorización menor	Media	CI + 1 reviewer + CODEOWNERS
Nueva funcionalidad	Baja	Plan aprobado + CI + 2 reviewers
Cambio en infraestructura environment gate	Muy baja	Plan aprobado + CI + 2 reviewers +
Migración de base de datos	Mínima	Solo con supervisión directa

Configuración del agente para generar artefactos inspeccionables

Los **artefactos inspeccionables** son los productos tangibles del trabajo del agente que pueden ser revisados con herramientas de desarrollo estándar:

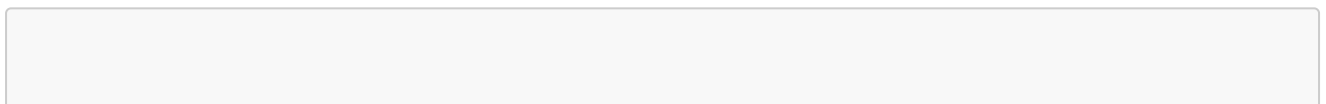
1. **Pull Requests como artefacto principal:**

- El diff del PR muestra exactamente qué cambió.
- Los comentarios del agente explican el por qué.
- El historial de commits muestra la secuencia de cambios.

2. **Logs de GitHub Actions:**

- Registro completo de la ejecución de CI/CD.
- Resultados de tests con detalle de fallos.
- Output de linters y herramientas de análisis.

3. **Artefactos de Actions (upload-artifact):**



```
# Ejemplo: guardar reportes generados por el agente como artefactos
- name: Upload agent report
  uses: actions/upload-artifact@v4
  with:
    name: agent-analysis-report
    path: |
      reports/coverage.html
      reports/lint-results.json
      reports/agent-decisions.md
```

4. Comentarios estructurados en PRs:

El agente debe documentar sus decisiones en el PR:

```
## Resumen de cambios del agente

### Decisiones tomadas
- Elegí usar `passport.js` sobre implementación manual porque...
- Creé un middleware separado en lugar de inline porque...

### Archivos modificados
- `src/auth/oauth.ts`: Nuevo módulo de autenticación
- `src/app.ts`: Integración del middleware (líneas 15-22)

### Tests agregados
- `tests/auth/oauth.test.ts`: 12 tests cubriendo flujos happy path y errores

### Riesgos conocidos
- El token refresh no está implementado (fuera de scope del issue)
```

5. Trazas de razonamiento del agente:

- GitHub Copilot mantiene el contexto de la sesión del agente.
- Las trazas son visibles en la interfaz de Copilot para entender el razonamiento.
- Los logs de la sesión del agente muestran qué herramientas usó y qué decisiones tomó.

Configurar la intervención humana sin ralentizar la entrega

El desafío es encontrar el balance entre **seguridad** (intervención humana) y **velocidad** (entrega continua). Estrategias para lograrlo:

1. Intervención asíncrona:

El agente trabaja y abre PRs. Los humanos revisan cuando tienen disponibilidad. No hay bloqueo síncrono.

Agente trabaja	Humano revisa	Agente continúa

(sin bloqueo)	PR Review	



Aprobado

2. Revisión por niveles (tiered review):

No todos los cambios requieren el mismo nivel de revisión:

```
# Ejemplo conceptual de política de revisión por niveles
bajo_riesgo:      # docs, typos, tests
  reviewers: 1
  auto_merge: true # tras CI verde + 1 aprobación
  timeout: 4h      # auto-recordatorio si no hay review

medio_riesgo:     # nuevas features, refactoros
  reviewers: 1
  codeowners: true
  auto_merge: false

alto_riesgo:      # infra, seguridad, APIs públicas
  reviewers: 2
  codeowners: true
  plan_required: true
  environment_gate: true
```

3. Notificaciones inteligentes:

- Configurar notificaciones de GitHub para PRs del agente.
- Usar etiquetas (`agent-generated`, `needs-review`) para filtrar.
- Integrar con Slack/Teams para alertas inmediatas en PRs críticos.

4. Auto-merge con condiciones:

Para cambios de bajo riesgo, configurar auto-merge cuando se cumplan todas las condiciones:

```
# Branch protection con auto-merge habilitado
auto_merge:
  enabled: true
  conditions:
    - status_checks: [CI/build, CI/test, CI/lint] # todos verdes
    - approved_reviews: 1 # mínimo 1 aprobación
    - labels: [low-risk, agent-generated] # etiquetas correctas
```

5. Uso de CODEOWNERS para enrutamiento automático:

```
# .github/CODEOWNERS
# Los dueños de cada área son notificados automáticamente
```

```
# cuando el agente toca archivos en su dominio
```

```
/src/auth/          @security-team  
/src/api/           @backend-team  
/src/frontend/      @frontend-team  
/infrastructure/    @devops-team  
/docs/             @tech-writers
```

Esto asegura que el **experto correcto** revise cada cambio del agente, sin necesidad de asignación manual.

Aplicación práctica en GitHub

Configuración completa de un repositorio para agentes observables y controlados

```
Repositorio: mi-proyecto  
|  
├── .github/  
│   ├── copilot-instructions.md    ← Instrucciones para el agente  
│   ├── CODEOWNERS                ← Enrutamiento de revisiones  
│   ├── workflows/  
│   │   ├── ci.yml                ← CI estándar (build, test, lint)  
│   │   ├── agent-checks.yml      ← Checks adicionales para PRs del agente  
│   │   └── deploy.yml            ← Deploy con environment gates  
│   └── rulesets.json              ← Políticas de rama (exportadas)  
├── Settings (UI de GitHub)  
│   ├── Branches → Protection rules para main  
│   ├── Environments → "production" con required reviewers  
│   └── Rules → Rulesets para ramas copilot/*
```

Workflow de CI específico para trabajo del agente

```
# .github/workflows/agent-checks.yml  
name: Agent Work Validation  
  
on:  
  pull_request:  
    branches: [main]  
  
jobs:  
  validate-agent-work:  
    # Solo ejecutar si el PR fue creado por el agente  
    if: contains(github.event.pull_request.labels.*.name, 'agent-generated')  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4
```

```

- name: Verificar que el plan existe
  run: |
    # Verificar que el agente documentó su plan en el PR
    echo "Verificando documentación del agente..."

- name: Ejecutar tests
  run: npm test

- name: Análisis de seguridad
  run: npm audit

- name: Verificar cobertura de tests
  run: |
    npm run test:coverage
    # Fallar si la cobertura baja del umbral

- name: Upload reportes
  uses: actions/upload-artifact@v4
  with:
    name: agent-validation-report
    path: coverage/

```

Puntos clave para el examen

- ☒ La **observabilidad** se logra principalmente a través de PRs, commits, logs de Actions y artefactos.
- ☒ El grado de autonomía debe ser **proporcional al riesgo** e **inversamente proporcional al impacto** del error.
- ☒ Los **guardrails** principales en GitHub son: branch protection, CODEOWNERS, required reviews, required status checks y rulesets.
- ☒ Los artefactos inspeccionables deben ser legibles con **herramientas estándar de desarrollo** (diff viewers, browsers, IDEs).
- ☒ La intervención humana se configura como **asíncrona** para no bloquear la entrega: el agente trabaja, el humano revisa cuando puede.
- ☒ **Auto-merge con condiciones** es un mecanismo válido para cambios de bajo riesgo con CI verde y aprobación.
- ☒ **CODEOWNERS** permite enrutar automáticamente la revisión al experto correcto según los archivos modificados.
- ☒ La trazabilidad completa del agente debe permitir responder: **qué hizo, por qué lo hizo y cuál fue el resultado**.
- ☒ El principio de **mínimo privilegio** aplica a los permisos del agente: solo los permisos necesarios para la tarea.

Resumen de la Sección 1

Concepto	Descripción clave
----------	-------------------

Concepto	Descripción clave
Agentes como contribuidores	Tratar al agente como un dev junior: su trabajo pasa por PR y review
GitHub como plano de control	GitHub es el sistema de registro para toda actividad del agente
Plan → Act → Evaluate	Ciclo fundamental: planificar, actuar, evaluar. Nunca omitir fases
Antipatrones	Sobre-autonomía, sin validación, omitir planificación, agente infalible
Entradas/Salidas	Issues claros → PRs con código + tests. Criterio de éxito: CI verde + review aprobado
Separación plan/ejecución	El agente no actúa hasta que el plan esté aprobado por un humano
Guardrails	Branch protection, CODEOWNERS, required reviews, rulesets, environments
Observabilidad	PRs, commits, Actions logs, artefactos, comentarios estructurados
Autonomía proporcional	Más riesgo = menos autonomía. Más cobertura de tests = más confianza
Intervención asíncrona	El humano revisa sin bloquear. Auto-merge para bajo riesgo

Recursos

Recurso	Enlace
Study Guide oficial GH-600	learn.microsoft.com
Foundations of Agentic AI in GitHub	Microsoft Learn
Designing Agent Architecture and SDLC Integration	Microsoft Learn
GitHub Docs — Copilot Coding Agent	docs.github.com
GitHub Docs — Repository Rulesets	docs.github.com
GitHub Docs — CODEOWNERS	docs.github.com
GitHub Docs — Branch Protection Rules	docs.github.com
GitHub Docs — GitHub Actions Environments	docs.github.com
GitHub Docs — Copilot Instructions	docs.github.com

Nota: Esta guía está basada en los objetivos publicados del examen GH-600 (Beta). Consulta siempre la guía de estudio oficial para verificar cambios en el contenido evaluado.